

# 第 27 讲

## PyTorch Tensor

### 从 NumPy 数组进入自动微分框架

《面向 LLM / Agent Systems 科研的深度学习零基础课》

**【本讲定位】**第 26 讲已经用 NumPy 手写了 Forward、Loss、Backward 和 Training Loop。现在开始进入 PyTorch。第一步不是急着学习 `nn.Linear` 或 `loss.backward()`，而是先弄清最基础的数据对象：`torch.Tensor`。你需要知道它和 NumPy `ndarray` 哪里相同、哪里不同，以及 `dtype`、`device`、`requires_grad` 为什么会影响后续训练。

## 学习目标

学完本讲，你应该能够：

1. 用自己的话解释 `torch.Tensor` 是什么；
2. 读懂 Tensor 的 `shape` / `ndim` / `dtype` / `device` / `requires_grad`；
3. 理解 NumPy 与 PyTorch 在索引、切片、Shape、矩阵乘法上的连续性；
4. 区分逐元素乘法 `*` 和矩阵乘法 `@`；
5. 知道为什么深度学习代码必须关注 `dtype`；
6. 理解 Tensor 为什么可以放在 CPU 或 GPU；
7. 区分 `torch.tensor()` 与 `torch.from_numpy()` 的常见内存行为；
8. 理解 `requires_grad=True` 只是“开始记录梯度依赖”，还不等于已经完成 Backpropagation；
9. 看到 LLM 中的  $(B, T, D)$  Tensor 时，继续使用前面已经建立的 Shape 直觉。

## 1 【原理】为什么已经会 NumPy 还要学 PyTorch Tensor?

NumPy 已经能做：

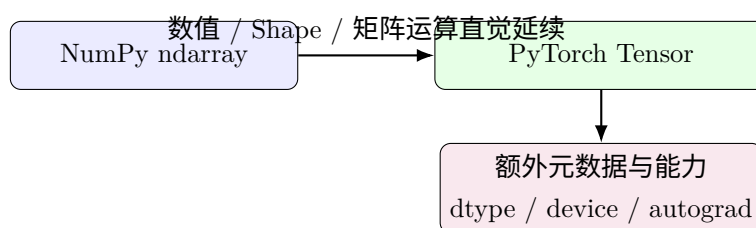
- 多维数组；
- Shape；
- 矩阵乘法；

- 广播；
- 基本数值计算。

所以从“装数字”的角度，PyTorch Tensor 并不是完全新的东西。

但是训练神经网络还需要额外能力：

1. 把数据和参数放到 GPU；
2. 自动记录计算依赖；
3. 自动计算梯度；
4. 与神经网络 Layer、Optimizer、Dataset 等组件协作。

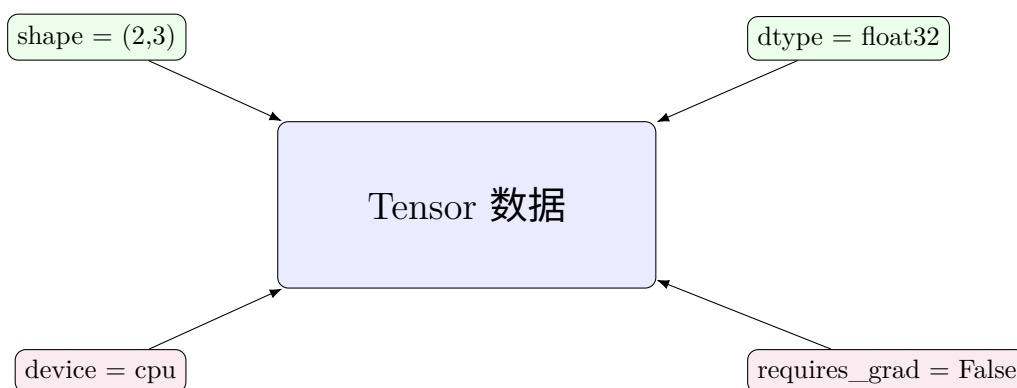


可以把 `torch.Tensor` 理解为：“带有深度学习运行信息的多维数字容器”。数值本身仍然是数字，但 Tensor 还知道“我是什么类型、我在哪个设备、我要不要参与梯度计算”。

## 2 【通俗解释】Tensor 像一箱数字，但箱子上多了标签

如果 NumPy 数组是一箱数字，那么 PyTorch Tensor 除了箱子里的数字，还贴着几张标签：

- `shape`：这箱数字按什么结构摆放；
- `dtype`：每个数字用什么精度保存；
- `device`：数字现在存放在 CPU 还是 GPU；
- `requires_grad`：是否需要为后续自动求导记录计算关系。



### 3 【具体数字例子】同一组数字：NumPy 与 PyTorch

我们先使用完全相同的数据：

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

它的 Shape 是：

(2, 3)

NumPy：

```
np_x = np.array([[1., 2., 3.],  
                [4., 5., 6.]], dtype=np.float32)
```

PyTorch：

```
torch_x = torch.tensor([[1., 2., 3.],  
                        [4., 5., 6.]], dtype=torch.float32)
```

两者都表示同一个  $2 \times 3$  数字表。PyTorch 额外可以告诉我们：

```
torch_x.shape  
# torch.Size([2, 3])  
  
torch_x.dtype  
# torch.float32  
  
torch_x.device  
# cpu  
  
torch_x.requires_grad  
# False
```

### 4 【必要概念】Tensor 的五个核心属性

#### 1. values：里面具体是什么数字

这仍然是最基础的内容。Tensor 不是抽象符号，它内部就是实际数值。

#### 2. shape：每个轴有多长

例如：

(2, 3, 4)

表示三维 Tensor，有三个轴，长度分别是 2、3、4。

### 3. ndim: 有几个轴

$$\text{shape} = (2, 3, 4) \Rightarrow \text{ndim} = 3$$

### 4. dtype: 数字怎样编码

常见类型包括:

- torch.float32: 常见训练浮点类型;
- torch.float64: 更高精度, 但更占内存、计算更慢;
- torch.float16 / bfloat16: 现代大模型训练与推理中常见的低精度浮点;
- torch.int64: Token ID、类别标签等常见整数类型。

### 5. device: 数字放在哪里

例如:

```
tensor.device
# cpu
```

如果有 CUDA GPU:

```
tensor = tensor.to("cuda")
```

## 5 【Shape 复习】PyTorch 索引规则基本延续 NumPy

给定:

$$X.\text{shape} = (2, 3)$$

| 代码                     | 结果 Shape            | 原因              |
|------------------------|---------------------|-----------------|
| <code>X[0]</code>      | <code>(3,)</code>   | 用整数索引, 第 0 维被移除 |
| <code>X[0:1]</code>    | <code>(1, 3)</code> | 用切片, 第 0 维被保留   |
| <code>X[:, 1]</code>   | <code>(2,)</code>   | 第 1 维用整数索引, 被移除 |
| <code>X[:, 1:2]</code> | <code>(2, 1)</code> | 切片保留第 1 维       |

从 NumPy 进入 PyTorch 后, 不需要把 Shape 直觉推翻重学。第 1-6 讲建立的大部分索引、切片、reshape、矩阵乘法规则仍然成立。

## 6 【重要区别】\* 和 @ 不是一回事

假设：

$$A.shape = (2, 3), \quad B.shape = (2, 3)$$

$A * B$  是逐元素乘法：

$$(A * B)_{ij} = A_{ij} B_{ij}$$

输出仍然是：

$$(2, 3)$$

而如果：

$$W.shape = (3, 2)$$

那么：

$$A @ W : (2, 3) @ (3, 2) \rightarrow (2, 2)$$

这才是矩阵乘法。

**【常见错误】**深度学习初学者经常把 \* 和 @ 混淆。Linear、Q/K/V projection、Attention 中大量核心计算依赖矩阵乘法，而不是逐元素乘法。

## 7 【dtype】为什么实验里不能忽略数据类型？

同样写：

```
[1.0, 2.0, 3.0]
```

NumPy 和 PyTorch 的默认浮点 dtype 可能不同。

在本讲环境中：

- NumPy 默认得到 float64；
- PyTorch 默认得到 float32。

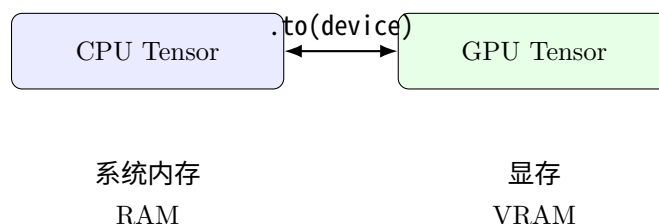
这会影响：

- 内存占用；
- 运算速度；
- 数值精度；
- 某些算子能否匹配；
- GPU 是否支持高效计算。

在 LLM 中，模型参数可能是 FP32、FP16、BF16、INT8、INT4 等。以后学习 mixed precision 和 quantization 时，dtype 会从“一个小标签”变成非常重要的系统变量。

## 8 【device】CPU 与 GPU 到底意味着什么？

Tensor 不只是“一个数学数组”，它还必须实际存放在某块硬件内存里。



如果当前机器没有 CUDA：

```
torch.cuda.is_available()
# False
```

代码仍然可以在 CPU 正常运行。

**【常见错误】参与同一个运算的 Tensor 通常需要在兼容的 device 上。一个在 CPU、一个在 CUDA GPU，直接做运算往往会报 device mismatch。**

## 9 【NumPy 和 Tensor 转换】复制还是共享内存？

两种常见写法：

```
t_shared = torch.from_numpy(a)
t_copy = torch.tensor(a)
```

在常见 CPU 场景中：

- `torch.from_numpy(a)` 通常和 NumPy 数组共享底层内存；
- `torch.tensor(a)` 通常创建一个新的副本。

实验中：

```
a = [1, 2, 3]
a[0] = 99
```

结果：

- `from_numpy` 创建的 Tensor 同步变成 99；
- `torch.tensor` 创建的副本仍然是 1。

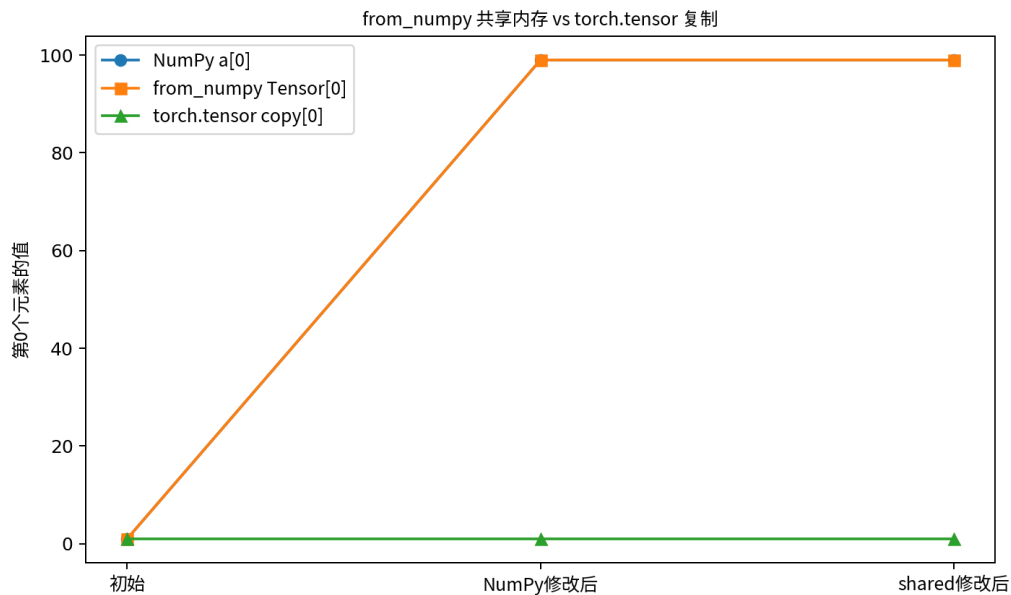


图 1: from\_numpy 共享内存与 torch.tensor 复制的实验对照

**【科研提示】** 共享内存很高效，但也可能制造“我只改了一个对象，另一个对象为什么也变了”的隐蔽 Bug。实验代码中涉及 NumPy/PyTorch 转换时，要明确你需要的是共享还是复制。

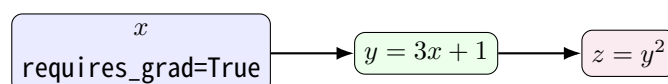
## 10 【Autograd 预览】requires\_grad 到底是什么意思？

现在只做预览，不正式学习 backward()。

```
x = torch.tensor([2.0, -1.0], requires_grad=True)
y = x * 3.0 + 1.0
z = y ** 2
```

此时：

- `x.requires_grad = True`;
- 用 `x` 计算出来的 `y, z` 会继续记录梯度依赖；
- `z.grad_fn` 会指向产生它的 backward 节点；
- 但还没有执行 `backward()`，所以 `x.grad` 仍然是 `None`。



PyTorch 记录  
“谁由谁计算而来”

**requires\_grad=True** 的含义不是“已经有梯度”，而是：从这里开始，PyTorch 需要记录与梯度计算相关的计算依赖。真正把梯度算出来，是第 29 讲的 `loss.backward()`。

## leaf Tensor 是什么？

在最常见的训练场景中，模型参数属于 leaf Tensor：它们是计算图的“输入参数”，不是由别的需要梯度的 Tensor 运算得到的。

例如上面的  $x$ ：

```
x.is_leaf
# True

y.is_leaf
# False
```

## 11 【推理预览】torch.no\_grad() 为什么存在？

推理时，我们通常只需要 Forward，不需要 Backward。

```
with torch.no_grad():
    y = model(x)
```

直觉上就是告诉 PyTorch：

“这一段只需要算结果，不需要为将来的反向传播保存梯度依赖。”

这样通常可以减少额外的内存和计算开销。

## 12 【与 LLM 连接】 $(B, T, D)$ 在 PyTorch 里没有换一套数学

假设：

$$X.shape = (B, T, D) = (2, 3, 4)$$

以及：

$$W.shape = (D, O) = (4, 5)$$

那么：

$$Y = X @ W$$

得到：

$$Y.shape = (2, 3, 5)$$

也就是：

$$(B, T, D) @ (D, O) \rightarrow (B, T, O)$$

这和第 6 讲完全相同，只是现在对象从 NumPy array 换成了 torch.Tensor。

以后 Transformer 中：

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

本质仍然是在 Tensor 上执行同一类矩阵运算。

## 13 【代码实验】建议按什么顺序运行？

本讲代码文件：

```
lesson27_pytorch_tensor_experiment.py
```

建议不要直接看完输出，而是每个实验先预测：

1. Shape 会是多少？
2. dtype 会是什么？
3. 是否共享内存？
4. requires\_grad 会不会继续传播？
5.  $(B, T, D) @ (D, O)$  输出 Shape 是多少？

然后再运行代码对照。

## 14 【常见误区】

1. “Tensor 就是 GPU 数据。” 错误。Tensor 完全可以在 CPU；GPU 只是 device 的一种。
2. “requires\_grad=True 就已经算出了梯度。” 错误。它只是开启追踪；真正梯度通常在 backward 后出现。
3. “PyTorch 和 NumPy 的 Shape 规则完全不同。” 错误。大量基础规则高度一致。
4. “float32 / float64 随便混用都没有影响。” 错误。dtype 会影响内存、速度、精度和算子兼容。
5. “from\_numpy 一定复制数据。” 错误。常见 CPU 场景下通常共享底层内存。
6. “Tensor 转 NumPy 直接.numpy() 永远可以。” 不总是。参与梯度跟踪或位于 GPU 的 Tensor 通常需要先 detach().cpu()。

## 15 【与科研的关系】为什么这些“工程细节”值得学？

以后看论文代码，经常会遇到：

```
x = x.to(device)
x = x.float()
with torch.no_grad():
    ...
features = tensor.detach().cpu().numpy()
```

如果不理解 Tensor 元数据，很容易把这些行当作“框架语法噪声”。实际上它们决定了：

- 数据在哪里计算；
- 用什么精度计算；
- 是否建立梯度图；
- 能否安全导出到 NumPy；
- 实验是否会报 dtype/device mismatch。

在 Agent Systems 研究里，你通常不会自己训练超大基础模型，但会经常读取 embedding、hidden states、logits、reward、evaluation scores。这些对象几乎都会以 Tensor 的形式出现。Tensor 是连接“模型内部计算”和“Agent 系统实验代码”的基础数据结构。

## 16 本讲最低验收标准

如果你能不看答案解析下面 6 句话，本讲就算掌握：

1. torch.Tensor 与 NumPy ndarray 的核心联系是什么？
2. shape / dtype / device / requires\_grad 分别是什么意思？
3. 为什么  $A * B$  和  $A @ B$  不能混淆？
4. 为什么 from\_numpy 可能带来“改一个，另一个也变”的现象？
5. requires\_grad=True 为什么不等于 grad 已经有值？
6. 为什么  $(B, T, D) @ (D, O) \rightarrow (B, T, O)$  在 PyTorch 里仍然成立？

## 下一讲

第 28 讲：PyTorch 中的 Linear 与 Activation。我们会把之前自己写的

$$Y = XW + b$$

和

$$A = ReLU(Y)$$

正式替换成：

```
nn.Linear
nn.ReLU
```

并直接检查 PyTorch 内部到底保存了哪些 Weight、Bias，以及它们和我们前面手写矩阵之间是什么关系。